



Domain Driven Design - Java

Prof. Gilberto Alexandre das Neves
profgilberto.neves@fiap.com.br

Collections Framework

O **Collections Framework** em Java é um conjunto de interfaces e classes que fornecem uma arquitetura padrão para armazenar e manipular grupos de dados como listas, conjuntos, pilhas, filas, entre outros. Ele faz parte do pacote **java.util** e foi introduzido no Java 2 Standard Edition (J2SE) para resolver a necessidade de uma maneira consistente e eficiente de lidar com coleções de objetos.

Interfaces:

- **Collection:** A raiz da hierarquia de interfaces. Ela define operações básicas como add, remove, size, etc.
- **List:** Estende *Collection* e representa uma coleção ordenada (sequencial) de elementos. Permite elementos duplicados.
- **Set:** Também estende *Collection*, mas representa uma coleção de elementos únicos, sem duplicatas.
- **Map:** Embora não estenda *Collection*, representa uma coleção de pares chave-valor. Cada chave deve ser única.

As principais implementações fornecidas pelo framework incluem:

- **ArrayList**: Implementa a interface **List** usando um *array* dinâmico. É ideal quando o número de elementos é variável.
- **HashSet**: Implementa a interface **Set** e usa uma tabela de *hash* para armazenar os elementos, garantindo a unicidade dos mesmos.
- **HashMap**: Implementa a interface **Map** usando uma tabela de *hash* para armazenar pares chave-valor.

O framework também inclui várias classes de utilidade na classe **Collections**, que fornece algoritmos como ordenação, pesquisa e manipulação de coleções. Alguns métodos importantes incluem:

- **sort()**: Para ordenar listas.
- **reverse()**: Para reverter a ordem dos elementos em uma lista.
- **shuffle()**: Para embaralhar os elementos de uma lista.

Exemplo HashSet

```
6  ▶ public class SorteioHashSet {
7  ▶     public static void main(String[] args) {
8      HashSet<Integer> numerosSorteados = new HashSet<Integer>();
9      Random random = new Random();
10     do {
11         JOptionPane.showMessageDialog(null, "Confira a seguir o resultados do sorteio da Loteria
12             da Sorte Premiada!", "Loteria", JOptionPane.INFORMATION_MESSAGE);
13         while (numerosSorteados.size() < 6) {
14             int numero = random.nextInt(59) + 1;
15             numerosSorteados.add(numero);
16         }
17         ArrayList<Integer> resultadoDoSorteio = new ArrayList<Integer>(numerosSorteados);
18         Collections.sort(resultadoDoSorteio);
19         JOptionPane.showMessageDialog(null, "Os números sorteados são:\n" + resultadoDoSorteio);
20         numerosSorteados.clear();
21     } while (JOptionPane.showConfirmDialog(null, "Deseja continuar?", "Atenção", JOptionPane
22         .YES_NO_OPTION, JOptionPane.QUESTION_MESSAGE) == 0);
23     JOptionPane.showMessageDialog(null, "Fim de Programa!", "Adeus", JOptionPane.WARNING_MESSAGE);
24 }
```

Navegando pelo HashMap

Para navegar por um **HashMap** utilizamos o comando **for-each**. Exemplo:

```
6 ▶ public class NavegaHashMap {
7 ▶     public static void main(String[] args) {
8         HashMap<String, Integer> pessoa = new HashMap<String, Integer>();
9         pessoa.put("Astrogildo", 18);
10        pessoa.put("Berisvaldo", 25);
11        pessoa.put("Gumercendo", 18);
12        for (Map.Entry<String, Integer> valor : pessoa.entrySet()) {
13            System.out.println("Chave: " + valor.getKey() + "\tValor: " + valor.getValue());
14        }
15    }
16 }
```

Resultado:

```
Chave: Gumercendo    Valor: 18
Chave: Berisvaldo    Valor: 25
Chave: Astrogildo    Valor: 18

Process finished with exit code 0
```

Exemplo HashMap

```
6 > public class ExemploHashMap {
7 >     public static void main(String[] args) {
8         HashMap<String, String> mapa = new HashMap<String, String>();
9         do {
10             try {
11                 String sigla, estado;
12                 do {
13                     sigla = JOptionPane.showInputDialog("Digite a UF (Unidade Federal) de um estado  
ou digite \"fim\" para encerrar").toUpperCase();
14                     if (!sigla.equals("FIM")) {
15                         estado = JOptionPane.showInputDialog("Digite o nome completo do estado  
informado anteriormente");
16                         if (mapa.containsKey(sigla)) {
17                             JOptionPane.showMessageDialog(null, "Este estado já foi cadastrado!");
18                         } else {
19                             mapa.put(sigla, estado);
20                         }
21                     }
22                 } while (!sigla.equals("FIM"));
            }
        }
```

Exemplo HashMap

```
23 String escolha = JOptionPane.showInputDialog("Digite a UF de um estado a sua  
24         escolha").toUpperCase();  
25 if (mapa.containsKey(escolha)) {  
26     JOptionPane.showMessageDialog(null, "O nome completo deste estado é: " +  
27         mapa.get(escolha), "Nome do Estado", JOptionPane.INFORMATION_MESSAGE);  
28 } else {  
29     JOptionPane.showMessageDialog(null, "Estado não cadastrado!", "Atenção",  
30         JOptionPane.WARNING_MESSAGE);  
31 }  
32 } catch (Exception e) {  
33     JOptionPane.showMessageDialog(null, e.getMessage(), "Erro", JOptionPane  
34         .ERROR_MESSAGE);  
35 }  
36 }  
37 }  
38 }  
39 }  
40 }  
41 }  
42 }  
43 }  
44 }  
45 }  
46 }  
47 }  
48 }  
49 }  
50 }  
51 }  
52 }  
53 }  
54 }  
55 }
```


Praticando...

Implemente a classe **PokemonHashMap** com o método **main**. Veja a seguir as orientações:

1. Peça para o usuário digitar o nome e o tipo de pokémons e armazene em um objeto da classe **HashMap** (armazene somente pokémons novos, não permita repetições). Quando quiser parar ele deve digitar “**fim**”.
2. Peça para o usuário digitar um **tipo** qualquer a escolha dele.
3. Percorra todo o **HashMap** em busca de todos os nomes de pokémons do tipo escolhido pelo usuário. Exiba na tela o **tipo** que ele escolheu e o **nome de todos** os pokémons daquele tipo.
4. Perguntar ao usuário se deseja continuar (em caso afirmativo repetir os passos de 1 a 4 novamente).
5. Em caso negativo, exibir uma mensagem se despedindo do usuário e encerre o programa.

Questionário

1. Observe o código abaixo e indique qual valor será exibido após executar essa classe?

```
HashSet<String> conjunto = new HashSet<>();  
conjunto.add("Maçã");  
conjunto.add("Banana");  
conjunto.add("Laranja");  
conjunto.add("Maçã");  
  
System.out.println(conjunto.size());
```

- a) 4
- b) 3
- c) 2
- d) 1

2. Observe o código abaixo e indique qual valor será exibido após executar essa classe?

```
HashMap<String, Integer> mapa = new HashMap<>();  
mapa.put("X", 10);  
mapa.put("Y", 20);  
mapa.put("Z", 30);  
  
System.out.println(mapa.size());  
mapa.clear();  
System.out.println(mapa.size());
```

- a) 3, 3
- b) 3, 0
- c) 0, 3
- d) 0, 0



Java como programar. Paul Deitel e Harvey Deitel. Pearson, 2011.

Java 8 – Ensino Didático : Desenvolvimento e Implementação de Aplicações. Sérgio Furgeri. Editora Érica, 2015.

Até breve!

Questionário

1. Observe o código abaixo e indique qual valor será exibido após executar essa classe?

- a) 4
- ☒ b) 3
- c) 2
- d) 1

```
HashSet<String> conjunto = new HashSet<>();  
conjunto.add("Maçã");  
conjunto.add("Banana");  
conjunto.add("Laranja");  
conjunto.add("Maçã");  
  
System.out.println(conjunto.size());
```

2. Observe o código abaixo e indique qual valor será exibido após executar essa classe?

- a) 3, 3
- ☒ b) 3, 0
- c) 0, 3
- d) 0, 0

```
HashMap<String, Integer> mapa = new HashMap<>();  
mapa.put("X", 10);  
mapa.put("Y", 20);  
mapa.put("Z", 30);  
  
System.out.println(mapa.size());  
mapa.clear();  
System.out.println(mapa.size());
```